

1 Introduction

Tomasulo Algorithm Tomasulo's algorithm is used for dynamic scheduling of instructions that allows out-of-order execution and enables more efficient use of multiple execution units.

In this Assignment I have done following,

- 1. Modified the existing python implementation of tomasulo algorithm to support issuing multiple instructions, write-back multiple completed instructions and committing multiple instructions. (supports variable issue width)
- 2. I have tested my modification with a set of hard-ware configurations and sample test-cases.

What Works

- Supports for multiple issue, write-back, and commit.
- Functional units are pipe-lined.
- Implemented in-order execution, with multiple issue.
- This implementation supports multiple functional units and multiple load-store units; thus, changing configuration with functional units and load-store units more than one will also work.

Limitations

- This implementation doesn't support the branch-predictions, so inputs with branch commands such as "beq" and "bne" will fail when the issue-width is greater than 1.
- Load store unit is not pipe-lined.

2 Code Changes

In this section, I will discuss the code changes I have made to existing tomasulo implementation. I have used the python-version of tomasulo algorithm and make the following changes,

- (1) I have added the support to issue multiple instructions in a cycle (support 1-4 issues per clock cycle). For this I have added an additional parameter to the following functions which provide the issue width,
 - “issue” in “issue.py” - to issue multiple issues, (the number of issues that can be issued in a cycle is determined by the issue-width.)

Initial Version

```
def issue(cycle, PC, instructions, ROB, size_ROB,
          rs_int_adder,
          rs_fp_adder,
          rs_fp_multi,
          ld_sd_queue, size_ld_sd_queue,
          rat_int, rat_fp):
    ...
```

My Version

```
def issue(cycle, PC, instructions, ROB, size_ROB,
          rs_int_adder,
          rs_fp_adder,
          rs_fp_multi,
          ld_sd_queue, size_ld_sd_queue,
          rat_int, rat_fp, issue_width):
    # fetch instruction same as issue-width
    for i in range(min(issue_width, len(instructions) - PC.PC)):
    ...
```

- “wb” in “wb.py” - to write-back completed issues,(the number of issues that can be write-back in a cycle is determined by the issue-width.)

Initial Version

```
def wb(cdb, rat_int, rat_fp,
      rs_int_adder, rs_fp_adder, rs_fp_multi,
      ld_sd_queue, ROB, cycle,
      results_buffer):
    # broadcast
    if cdb.valid == 1:
        broadcast(cdb, rat_int, rat_fp,
                  rs_int_adder, rs_fp_adder, rs_fp_multi,
                  ld_sd_queue, ROB, cycle)
        cdb.valid = 0
    # fetch data from results queue
    if cdb.valid == 0:
        if len(results_buffer)>0:
            cdb.valid = 1
            cdb.value = results_buffer[0].value
            cdb.dest_tag = results_buffer[0].dest_tag
            results_buffer.popleft()
```

My Version

```
def wb(cdb, rat_int, rat_fp,
      rs_int_adder, rs_fp_adder, rs_fp_multi,
      ld_sd_queue, ROB, cycle,
      results_buffer, issue_width):
    # broadcast
    bd_count = 0
    while len(results_buffer) > 0 and (bd_count < issue_width):
        cdb.valid = 1
        cdb.value = results_buffer[0].value
        cdb.dest_tag = results_buffer[0].dest_tag
        results_buffer.popleft()
        broadcast(cdb, rat_int, rat_fp,
```

```
        rs_int_adder, rs_fp_adder, rs_fp_multi,  
        ld_sd_queue, ROB, cycle)  
    bd_count += 1
```

- “commit” in “commit.py” - to commit, (the number of issues that can be committed in a cycle is determined by the issue-width.)

Initial Version

```
def commit(ROB, reg_int, reg_fp, cycle, instructions):  
    if len(ROB)>0:  
    ...
```

My Version

```
def commit(ROB, reg_int, reg_fp, cycle, instructions, issue_width):  
    for i in range(issue_width):  
        if len(ROB)>0:  
  
    ...
```

- (2) I have changed the function “fu_exe” in “exe.py” to avoid a functions being executed twice in the same cycle, when “fu_exe” is called for the second time in same cycle, when it tries to execute the waiting instructions.

Initial Version

```
def fu_exe(fu, fu_results, ROB, time_fu, cycle, PC):
    if len(fu)!=0:
        for element in fu:
            element.cycle += 1
    ...
```

My Version

```
# dictionary keep-track of the functional-unit called in a cycle
cycle_call_history = {}

def fu_exe(fu, fu_results, ROB, time_fu, cycle, PC):
    if len(fu) != 0:
        for element in fu:
            if (element, cycle) not in cycle_call_history.keys():
                element.cycle += 1
                cycle_call_history[(element, cycle)] = 1
    ....
```

- (3) I have changed the function “check_valid_ins_in_rs” in “exe.py” to make the in-order execution of the issued commands.

Initial Version

```
def check_valid_ins_in_rs(rs):
    index = -1
    if len(rs)!=0:
        for i in range(len(rs)):
            if (rs[i].valid_1st==1)&(rs[i].valid_2nd==1)&(rs[i].busy==1):
                index = i
                break
    return index
```

My Version

```
def check_valid_ins_in_rs(rs, ROB, cycle):
    index = -1
    if len(rs) != 0:
        possible_indices = []
        issue_cycles = []
        for i in range(len(rs)):
            if (rs[i].valid_1st == 1) & \
                (rs[i].valid_2nd == 1) \
                & (rs[i].busy == 1):
                possible_indices.append(i)
                index_r = find_ROB_entry(ROB, rs[i].dest_tag)
                issue_cycles.append(ROB[index_r].issue[0])

        if len(possible_indices) > 0:
            while len(issue_cycles) > 0:
                dependent = False
                min_index = issue_cycles.index(min(issue_cycles))
                index_t = possible_indices[min_index]
                index_p = find_ROB_entry(ROB, rs[index_t].dest_tag)
                selected_rob = ROB[index_p]
```

```
for i, rob_x in enumerate(ROB):
    if selected_rob.op == "Addi":
        dependent = False
        break
    else:
        if rob_x.PC < selected_rob.PC and \
            (selected_rob.tag_1st == rob_x.dest_tag or \
             selected_rob.tag_2nd == rob_x.dest_tag):
            if len(rob_x.cdb) == 0 or \
                (len(rob_x.cdb) == 1 and \
                 rob_x.cdb[0] == cycle):
                dependent = True
                break
if not dependent:
    index = possible_indices[min_index]
    break
else:
    if len(issue_cycles) >= 1:
        issue_cycles.pop(min_index)
        possible_indices.pop(min_index)
return index
```

- (4) I have changed the function “check_valid_ins_in_ldsd” in “exe.py” to make the in-order execution of the issued commands using load and store.

Initial Version

```
def check_valid_ins_in_ldsd(ldsd):
    index = -1
    if len(ldsd) != 0:
        for i in range(len(ldsd)):
            if (ldsd[i].valid == 1) & (ldsd[i].ready == 0):
                index = i
                break
    return index
```

My Version

```
def check_valid_ins_in_ldsd(ldsd, ROB, cycle):
    index = -1
    if len(ldsd) != 0:
        for i in range(len(ldsd)):
            if (ldsd[i].valid == 1) & (ldsd[i].ready == 0):
                dependent = False
                index_p = find_ROB_entry(ROB, ldsd[i].dest_tag)
                selected_rob = ROB[index_p]
                for x, rob_x in enumerate(ROB):
                    if rob_x.PC < selected_rob.PC \
                        and len(rob_x.cdb) == 1 and \
                        rob_x.cdb[0] == cycle:
                        if selected_rob.reg_tag == rob_x.dest_tag:
                            dependent = True
                            break
                if not dependent:
                    index = i
                    break
    return index
```

- (5) Finally, I have changed the order of calling pipeline stages to make the simulation work fine. (This is changed to simplify the multiple write-back and commit mechanism).
- swap the order of calling “exe”, “issue” functions to make the reservation station accessible at the same cycle when it is released.
 - swaps the loading memory instruction and doing memory stage. That is the other way round previously to avoid the same cycle execution and memory steps, now I changed since I changed the order of calling each stage in reverse order. Also swap the order of calling “mem”, “exe” in “main.py”.
 - swap the order of calling “cdb”, “commit” functions, and put them before “mem”, thus enabling in-order of execution and ensure that write-back and execution happen in separate cycles.
 - Finally, the functions are called in the following order in “main.py”, “commit”, “cdb”, “memory”, “exe” and “issue”. By this instruction completed in the previous cycle will only write-back in the current cycle and instructions committed in the current cycle if it write-back in the previous cycle.

3 Results

In this section, I have included the results of two different test cases with different architecture configurations and different lists of instructions.

3.1 TestCases

3.1.1 Test Case #01

Configuration of the Architecture

	# of rs	Cycles in EX	Cycles in Mem	# of FUs
Integer adder	4	1		1
FP adder	4	3		1
FP multiplier	4	5		1
Load/store unit	4	1	6	1

ROB entries = 64

R1=12, R2=32, F20=3.0

Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=4.0

List of Instructions

```

Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Add.d F4 F2 F4
Addi R1 R1 8
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Add.d F4 F2 F4
Addi R1 R1 8
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Add.d F4 F2 F4

```

	ISSUE	EXE	MEM	WB	COMMIT
Ld F1 0(R1)	[1]	[2, 2]	[3, 8]	[9]	[10]
Mult.d F2 F1 F0	[2]	[10, 14]	[]	[15]	[16]
Add.d F3 F3 F2	[3]	[16, 18]	[]	[19]	[20]
Mult.d F2 F1 F1	[4]	[11, 15]	[]	[17]	[21]
Add.d F4 F2 F4	[5]	[18, 20]	[]	[21]	[22]
Addi R1 R1 8	[6]	[7, 7]	[]	[8]	[23]
Ld F1 0(R1)	[7]	[9, 9]	[10, 15]	[16]	[24]
Mult.d F2 F1 F0	[8]	[17, 21]	[]	[23]	[25]
Add.d F3 F3 F2	[9]	[24, 26]	[]	[27]	[28]
Mult.d F2 F1 F1	[10]	[18, 22]	[]	[24]	[29]
Add.d F4 F2 F4	[11]	[25, 27]	[]	[28]	[30]
Addi R1 R1 8	[12]	[13, 13]	[]	[14]	[31]
Ld F1 0(R1)	[13]	[15, 15]	[16, 21]	[22]	[32]
Mult.d F2 F1 F0	[14]	[23, 27]	[]	[29]	[33]
Add.d F3 F3 F2	[16]	[30, 32]	[]	[33]	[34]
Mult.d F2 F1 F1	[17]	[24, 28]	[]	[30]	[35]
Add.d F4 F2 F4	[18]	[31, 33]	[]	[34]	[36]

Figure 1: Simulation results for the Test-case #01 with Issue-width #01

	ISSUE	EXE	MEM	WB	COMMIT
Ld F1 0(R1)	[1]	[2, 2]	[3, 8]	[9]	[10]
Mult.d F2 F1 F0	[1]	[10, 14]	[]	[15]	[16]
Add.d F3 F3 F2	[2]	[16, 18]	[]	[19]	[20]
Mult.d F2 F1 F1	[2]	[11, 15]	[]	[16]	[20]
Add.d F4 F2 F4	[3]	[17, 19]	[]	[20]	[21]
Addi R1 R1 8	[3]	[4, 4]	[]	[5]	[21]
Ld F1 0(R1)	[4]	[6, 6]	[9, 14]	[15]	[22]
Mult.d F2 F1 F0	[4]	[16, 20]	[]	[21]	[22]
Add.d F3 F3 F2	[5]	[22, 24]	[]	[25]	[26]
Mult.d F2 F1 F1	[5]	[17, 21]	[]	[22]	[26]
Add.d F4 F2 F4	[6]	[23, 25]	[]	[26]	[27]
Addi R1 R1 8	[6]	[7, 7]	[]	[8]	[27]
Ld F1 0(R1)	[7]	[9, 9]	[15, 20]	[21]	[28]
Mult.d F2 F1 F0	[10]	[22, 26]	[]	[27]	[28]
Add.d F3 F3 F2	[16]	[28, 30]	[]	[31]	[32]
Mult.d F2 F1 F1	[16]	[23, 27]	[]	[28]	[32]
Add.d F4 F2 F4	[17]	[29, 31]	[]	[32]	[33]

Figure 2: Simulation results for the Test-case #01 with Issue-width #02

	ISSUE	EXE	MEM	WB	COMMIT
Ld F1 0(R1)	[1]	[2, 2]	[3, 8]	[9]	[10]
Mult.d F2 F1 F0	[1]	[10, 14]	[]	[15]	[16]
Add.d F3 F3 F2	[1]	[16, 18]	[]	[19]	[20]
Mult.d F2 F1 F1	[2]	[11, 15]	[]	[16]	[20]
Add.d F4 F2 F4	[2]	[17, 19]	[]	[20]	[21]
Addi R1 R1 8	[2]	[3, 3]	[]	[4]	[21]
Ld F1 0(R1)	[3]	[5, 5]	[9, 14]	[15]	[21]
Mult.d F2 F1 F0	[3]	[16, 20]	[]	[21]	[22]
Add.d F3 F3 F2	[3]	[22, 24]	[]	[25]	[26]
Mult.d F2 F1 F1	[4]	[17, 21]	[]	[22]	[26]
Add.d F4 F2 F4	[4]	[23, 25]	[]	[26]	[27]
Addi R1 R1 8	[4]	[5, 5]	[]	[6]	[27]
Ld F1 0(R1)	[5]	[7, 7]	[15, 20]	[21]	[27]
Mult.d F2 F1 F0	[10]	[22, 26]	[]	[27]	[28]
Add.d F3 F3 F2	[16]	[28, 30]	[]	[31]	[32]
Mult.d F2 F1 F1	[16]	[23, 27]	[]	[28]	[32]
Add.d F4 F2 F4	[17]	[29, 31]	[]	[32]	[33]

Figure 3: Simulation results for the Test-case #01 with Issue-width #03

	ISSUE	EXE	MEM	WB	COMMIT
Ld F1 0(R1)	[1]	[2, 2]	[3, 8]	[9]	[10]
Mult.d F2 F1 F0	[1]	[10, 14]	[]	[15]	[16]
Add.d F3 F3 F2	[1]	[16, 18]	[]	[19]	[20]
Mult.d F2 F1 F1	[1]	[11, 15]	[]	[16]	[20]
Add.d F4 F2 F4	[2]	[17, 19]	[]	[20]	[21]
Addi R1 R1 8	[2]	[3, 3]	[]	[4]	[21]
Ld F1 0(R1)	[2]	[5, 5]	[9, 14]	[15]	[21]
Mult.d F2 F1 F0	[2]	[16, 20]	[]	[21]	[22]
Add.d F3 F3 F2	[3]	[22, 24]	[]	[25]	[26]
Mult.d F2 F1 F1	[3]	[17, 21]	[]	[22]	[26]
Add.d F4 F2 F4	[3]	[23, 25]	[]	[26]	[27]
Addi R1 R1 8	[3]	[4, 4]	[]	[5]	[27]
Ld F1 0(R1)	[4]	[6, 6]	[15, 20]	[21]	[27]
Mult.d F2 F1 F0	[10]	[22, 26]	[]	[27]	[28]
Add.d F3 F3 F2	[16]	[28, 30]	[]	[31]	[32]
Mult.d F2 F1 F1	[16]	[23, 27]	[]	[28]	[32]
Add.d F4 F2 F4	[17]	[29, 31]	[]	[32]	[33]

Figure 4: Simulation results for the Test-case #01 with Issue-width #04

3.1.2 Test Case #02

Configuration of the Architecture

	# of rs	Cycles in EX	Cycles in Mem	# of FUs
Integer adder	4	1		1
FP adder	3	3		2
FP multiplier	2	15		2
Load/store unit	4	1	5	2

ROB entries = 128

R1=12, R2=32, F20=3.0

Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=4.0

List of Instructions

```
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Ld F1 0(R1)
Addi R1 R1 4
Add.d F4 F2 F4
Addi R1 R1 8
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Add.d F4 F2 F4
```

	ISSUE	EXE	MEM	WB	COMMIT
Ld F1 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]
Mult.d F2 F1 F0	[2]	[9, 23]	[]	[24]	[25]
Add.d F3 F3 F2	[3]	[25, 27]	[]	[28]	[29]
Mult.d F2 F1 F1	[4]	[9, 23]	[]	[25]	[30]
Ld F1 0(R1)	[5]	[6, 6]	[7, 11]	[12]	[31]
Addi R1 R1 4	[6]	[7, 7]	[]	[9]	[32]
Add.d F4 F2 F4	[7]	[26, 28]	[]	[29]	[33]
Addi R1 R1 8	[8]	[9, 9]	[]	[10]	[34]
Ld F1 0(R1)	[9]	[11, 11]	[12, 16]	[17]	[35]
Mult.d F2 F1 F0	[10]	[18, 32]	[]	[33]	[36]
Add.d F3 F3 F2	[11]	[34, 36]	[]	[37]	[38]
Mult.d F2 F1 F1	[12]	[18, 32]	[]	[34]	[39]
Ld F1 0(R1)	[13]	[14, 14]	[15, 19]	[20]	[40]
Mult.d F2 F1 F0	[18]	[21, 35]	[]	[36]	[41]
Add.d F3 F3 F2	[25]	[38, 40]	[]	[41]	[42]
Mult.d F2 F1 F1	[26]	[27, 41]	[]	[42]	[43]
Add.d F4 F2 F4	[27]	[43, 45]	[]	[46]	[47]

Figure 5: Simulation results for the Test-case #02 with Issue-width #01

	ISSUE	EXE	MEM	WB	COMMIT
Ld F1 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]
Mult.d F2 F1 F0	[1]	[9, 23]	[]	[24]	[25]
Add.d F3 F3 F2	[2]	[25, 27]	[]	[28]	[29]
Mult.d F2 F1 F1	[2]	[9, 23]	[]	[24]	[29]
Ld F1 0(R1)	[3]	[4, 4]	[5, 9]	[10]	[30]
Addi R1 R1 4	[3]	[4, 4]	[]	[5]	[30]
Add.d F4 F2 F4	[4]	[25, 27]	[]	[28]	[31]
Addi R1 R1 8	[4]	[5, 5]	[]	[6]	[31]
Ld F1 0(R1)	[5]	[7, 7]	[8, 12]	[13]	[32]
Mult.d F2 F1 F0	[9]	[14, 28]	[]	[29]	[32]
Add.d F3 F3 F2	[9]	[30, 32]	[]	[33]	[34]
Mult.d F2 F1 F1	[10]	[14, 28]	[]	[29]	[34]
Ld F1 0(R1)	[10]	[11, 11]	[12, 16]	[17]	[35]
Mult.d F2 F1 F0	[14]	[18, 32]	[]	[33]	[35]
Add.d F3 F3 F2	[25]	[34, 36]	[]	[37]	[38]
Mult.d F2 F1 F1	[25]	[26, 40]	[]	[41]	[42]
Add.d F4 F2 F4	[26]	[42, 44]	[]	[45]	[46]

Figure 6: Simulation results for the Test-case #02 with Issue-width #02

	ISSUE	EXE	MEM	WB	COMMIT
Ld F1 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]
Mult.d F2 F1 F0	[1]	[9, 23]	[]	[24]	[25]
Add.d F3 F3 F2	[1]	[25, 27]	[]	[28]	[29]
Mult.d F2 F1 F1	[2]	[9, 23]	[]	[24]	[29]
Ld F1 0(R1)	[2]	[3, 3]	[4, 8]	[9]	[29]
Addi R1 R1 4	[2]	[3, 3]	[]	[4]	[30]
Add.d F4 F2 F4	[3]	[25, 27]	[]	[28]	[30]
Addi R1 R1 8	[3]	[4, 4]	[]	[5]	[30]
Ld F1 0(R1)	[3]	[6, 6]	[8, 12]	[13]	[31]
Mult.d F2 F1 F0	[9]	[14, 28]	[]	[29]	[31]
Add.d F3 F3 F2	[9]	[30, 32]	[]	[33]	[34]
Mult.d F2 F1 F1	[9]	[14, 28]	[]	[29]	[34]
Ld F1 0(R1)	[10]	[11, 11]	[12, 16]	[17]	[34]
Mult.d F2 F1 F0	[14]	[18, 32]	[]	[33]	[35]
Add.d F3 F3 F2	[25]	[34, 36]	[]	[37]	[38]
Mult.d F2 F1 F1	[25]	[26, 40]	[]	[41]	[42]
Add.d F4 F2 F4	[25]	[42, 44]	[]	[45]	[46]

Figure 7: Simulation results for the Test-case #02 with Issue-width #03

	ISSUE	EXE	MEM	WB	COMMIT
Ld F1 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]
Mult.d F2 F1 F0	[1]	[9, 23]	[]	[24]	[25]
Add.d F3 F3 F2	[1]	[25, 27]	[]	[28]	[29]
Mult.d F2 F1 F1	[1]	[9, 23]	[]	[24]	[29]
Ld F1 0(R1)	[2]	[3, 3]	[4, 8]	[9]	[29]
Addi R1 R1 4	[2]	[3, 3]	[]	[4]	[29]
Add.d F4 F2 F4	[2]	[25, 27]	[]	[28]	[30]
Addi R1 R1 8	[2]	[4, 4]	[]	[5]	[30]
Ld F1 0(R1)	[3]	[6, 6]	[8, 12]	[13]	[30]
Mult.d F2 F1 F0	[9]	[14, 28]	[]	[29]	[30]
Add.d F3 F3 F2	[9]	[30, 32]	[]	[33]	[34]
Mult.d F2 F1 F1	[9]	[14, 28]	[]	[29]	[34]
Ld F1 0(R1)	[9]	[10, 10]	[11, 15]	[16]	[34]
Mult.d F2 F1 F0	[14]	[17, 31]	[]	[32]	[34]
Add.d F3 F3 F2	[25]	[34, 36]	[]	[37]	[38]
Mult.d F2 F1 F1	[25]	[26, 40]	[]	[41]	[42]
Add.d F4 F2 F4	[25]	[42, 44]	[]	[45]	[46]

Figure 8: Simulation results for the Test-case #02 with Issue-width #04